

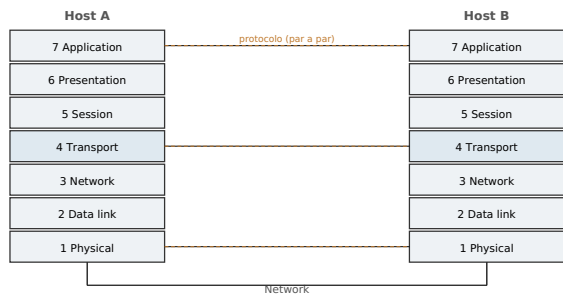
Comunicação

Resumo 10 de 14 · arquivo 4Comunicação · 61 imagens / 51 slides · Cap. 4 de van Steen & Tanenbaum

Aula longa e central: **como os processos de um sistema distribuído trocam informação**. Percorre os protocolos em camadas, os tipos de comunicação e quatro grandes famílias — **RPC** (chamada remota), **mensagens** (filas/brokers), **fluxos** (mídia contínua) e **multicast/epidêmicos** (disseminação em massa). Por ser extensa e muito visual, este resumo reproduz os diagramas do professor.

1. Protocolos em camadas

O modelo de referência é o **OSI**, de sete camadas: cada camada de uma máquina conversa com a mesma camada da outra por meio de um **protocolo**. As desvantagens de modelar tudo assim: só funciona com **passagem de mensagens**, arrasta funcionalidades desnecessárias e **viola a transparência de acesso**.



Modelo OSI de 7 camadas. Cada nível fala com o seu par via um protocolo; abaixo de tudo, a rede física.

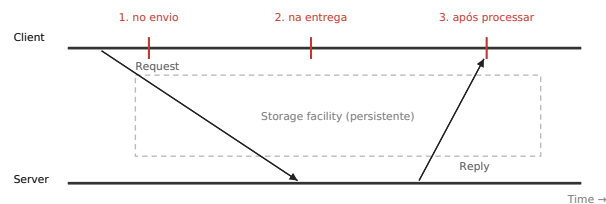
- **Camadas de baixo nível: física** (como os bits viram um quadro e são transmitidos), **enlace** (agrupa bits em quadros, faz detecção de erro e controle de fluxo) e **rede** (roteia pacotes). Em muitos SD, a interface mais baixa efetivamente usada é a de rede.
- **Camada de transporte:** onde estão as ferramentas realmente usadas. **TCP** — orientada a conexão, confiável, orientada a fluxo; **UDP** — datagramas não confiáveis (best-effort). O IP multicast é tratado como serviço padrão, mas contar com ele é arriscado.
- **Camada de middleware:** reúne protocolos comuns a muitas aplicações — comunicação, *(des)empacotamento (marshaling)*, gerenciamento de nomes, segurança e mecanismos de escalabilidade (replicação/cache). O que sobra são os protocolos específicos de cada aplicação.

2. Tipos de comunicação

Dois eixos independentes classificam a comunicação: **transiente x persistente** e **síncrona x assíncrona**.

- **Transiente:** a mensagem é descartada se não puder ser entregue naquele momento (não há armazenamento intermediário).
- **Persistente:** o sistema guarda a mensagem pelo tempo necessário até conseguir entregá-la.
- **Síncrona:** o remetente bloqueia até algum ponto de confirmação. **Assíncrona:** o remetente continua trabalhando imediatamente.

A sincronização pode ocorrer em três **pontos** distintos, e é isso que o diagrama abaixo mostra:

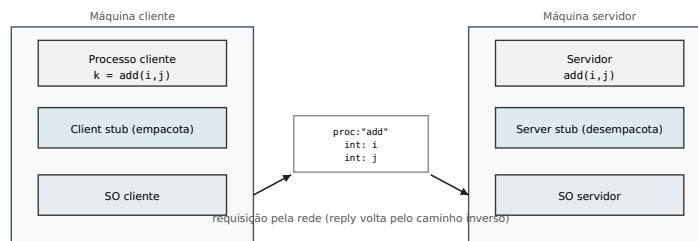


Os três pontos de sincronização: (1) no envio da requisição, (2) na entrega ao servidor e (3) após o processamento. O "storage facility" no meio representa o armazenamento da comunicação persistente.

Cliente/servidor é tipicamente **transiente síncrona**: ambos precisam estar ativos, o cliente envia a requisição e **bloqueia** até a resposta, e o servidor fica esperando requisições. As desvantagens da comunicação síncrona: o cliente não faz nada enquanto espera, as falhas precisam ser tratadas na hora (o cliente está bloqueado) e o modelo nem sempre é adequado (e-mail, news). Por isso existe o **middleware orientado a mensagens (MOM)**, que oferece comunicação **persistente assíncrona** por filas: o remetente não espera resposta imediata e o middleware normalmente garante tolerância a falhas.

3. RPC — chamada a procedimento remoto

A ideia: como um procedimento bem projetado é uma **caixa-preta** isolada, nada impede executá-lo em outra máquina — e a comunicação fica **escondida** atrás de uma chamada de procedimento comum. Em cada lado há um *stub* que cuida do empacotamento:



Mecanismo do RPC: a chamada local vira mensagem empacotada pelo client stub, cruza a rede, é desempacotada pelo server stub e executada como chamada local; o resultado volta pelo caminho inverso.

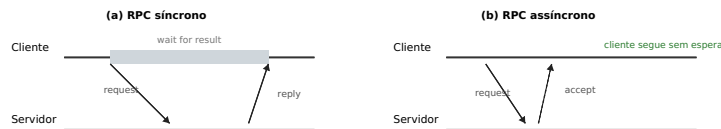
Os dez passos de uma chamada:

- 1. o processo cliente chama o *client stub*; 2. o stub monta a mensagem e chama o SO local; 3. o SO envia a mensagem ao SO remoto;
- 4. o SO remoto entrega a mensagem ao *server stub*; 5. o stub desempacota os parâmetros; 6. chama *add* localmente;
- 7. o server stub monta a mensagem de resposta; 8. o SO remoto a envia; 9. o SO do cliente a recebe; 10. o client stub desempacota o resultado e o devolve ao processo.

Passagem de parâmetros

Empacotar (**marshaling**) é mais que copiar valores: as máquinas podem ter **representações diferentes** (por exemplo, a ordem dos bytes), então cliente e servidor precisam concordar com uma **regra de codificação comum** — tanto para tipos básicos (inteiros, floats, caracteres) quanto complexos (vetores, unions). Valem duas suposições: semântica *copy in/copy out* e passagem apenas **por valor** (sem referências a dados globais). Consequência: **não há transparência total de acesso**. Um mecanismo de *referências remotas* (que podem ser passadas como parâmetro) melhora essa transparência.

Variações do RPC



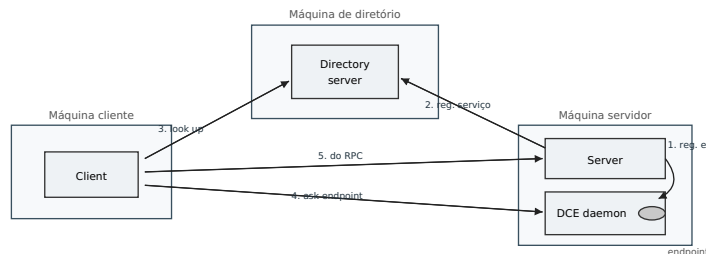
RPC síncrono (a): o cliente espera o resultado. RPC assíncrono (b): o servidor apenas confirma o recebimento e o cliente segue sem esperar o resultado.

Além do RPC clássico:

- RPC assíncrono:** livra-se do estrito requisição-resposta — o servidor confirma o recebimento e o cliente continua sem esperar o resultado.
- RPC síncrono diferido:** o cliente segue trabalhando e, quando o resultado fica pronto, o servidor o entrega por um **RPC de mão única** que interrompe o cliente (que confirma). Variante: o cliente faz *poll* (bloqueante ou não) para verificar se o resultado já saiu.

RPC na prática e vinculação (DCE)

Na prática escreve-se a interface numa **IDL (Interface Definition Language)**; seu compilador gera os *stubs* de cliente e servidor (e um cabeçalho comum), que são compilados e ligados a uma *runtime library*. Antes de chamar, o cliente precisa resolver dois problemas — **achar a máquina** do servidor e **achar o serviço**. A **vinculação cliente-servidor** do DCE resolve isso com um *directory server* e um *DCE daemon* (que mantém a *endpoint table*):

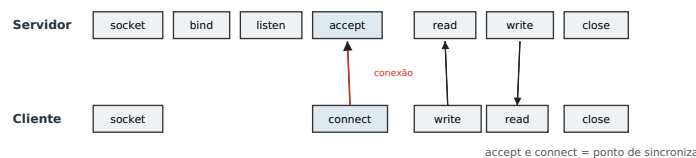


Vinculação DCE: o servidor (1) registra seu endpoint no daemon e (2) registra o serviço no diretório; o cliente (3) faz lookup no diretório, (4) pede o endpoint ao daemon e (5) faz o RPC.

4. Comunicação orientada a mensagens

Mensagens transientes: sockets

A interface **Berkeley sockets** tem oito primitivas: **SOCKET** (cria o ponto de comunicação), **BIND** (associa um endereço local), **LISTEN** (anuncia disposição de aceitar N conexões), **ACCEPT** (bloqueia até chegar um pedido), **CONNECT** (tenta conectar), **SEND/RECEIVE** (troca de dados) e **CLOSE** (encerra). O fluxo típico:



Sockets: o servidor faz socket→bind→listen→accept e bloqueia; o cliente faz socket→connect; em seguida trocam dados por read/write e encerram com close.

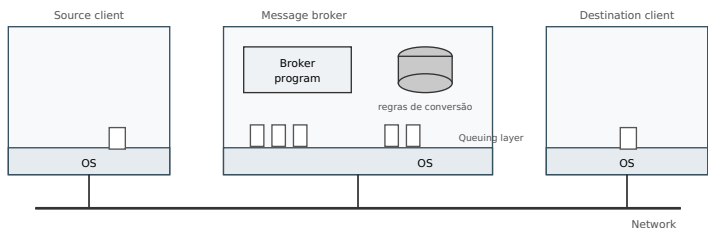
MOM: filas de mensagens

O MOM oferece comunicação **assíncrona e persistente** apoiada em **filas** (que correspondem a buffers nos servidores de comunicação). Quatro operações básicas:

- PUT — adiciona uma mensagem à fila indicada;
- GET — **bloqueia** até a fila ter mensagem e remove a primeira;
- POLL — verifica a fila e remove a primeira, mas **nunca bloqueia**;
- NOTIFY — instala um *tratador* chamado sempre que uma mensagem chega à fila.

Message broker

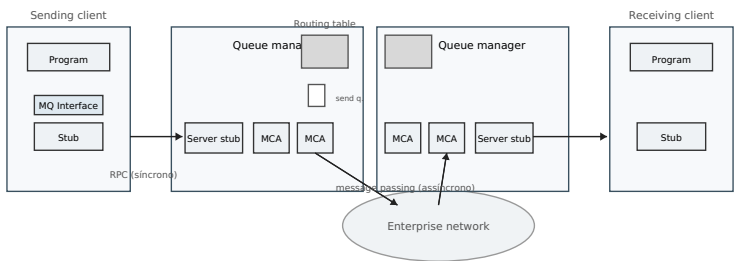
As filas assumem um **protocolo/formato comum** de mensagem. Quando as aplicações são heterogêneas, um **message broker** centraliza essa diferença: **transforma** as mensagens para o formato apropriado (usando um repositório de regras de conversão), frequentemente atua como *application gateway* e pode **rotear por conteúdo** — base da Integração de Aplicações Empresariais (EAI).



Message broker: entre o cliente de origem e o de destino, o broker converte formatos (repositório de regras de conversão) e gerencia as filas (queuing layer), tudo sobre a rede.

Exemplo: IBM Websphere MQ

No **IBM Websphere MQ**, mensagens específicas da aplicação são colocadas e retiradas de filas controladas por um **queue manager**. Um processo só coloca mensagens em filas *locais* (via RPC síncrono); a transferência entre filas de processos diferentes exige um **canal**, com um **agente de canal (MCA)** em cada ponta — que configura o canal (ex.: TCP/IP), (des)empacota e envia/recebe os pacotes. Pontos importantes: os **canais são unidirecionais**, os agentes são iniciados automaticamente quando chega mensagem, pode-se montar **redes de queue managers**, e as **rotas são configuradas manualmente** pelo administrador (usando nomes lógicos resolvidos por tabelas de rota/alias).



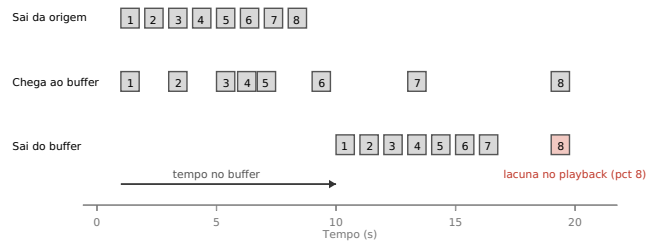
IBM Websphere MQ: o cliente fala com seu queue manager por RPC síncrono (local); entre queue managers, os agentes MCA transferem mensagens por canais unidirecionais (assíncrono) pela enterprise network.

5. Comunicação orientada a fluxo (streams)

Todas as ferramentas anteriores são **discretas**: úteis para trocas de informação independentes do tempo. A **mídia contínua**, ao contrário, depende do tempo entre seus valores — áudio, vídeo, animações, dados de sensores. Os **modos de transmissão** diferem nas garantias temporais: **assíncrono** (sem restrição de quando entregar), **síncrono** (atraso máximo por pacote) e **isócrono** (atraso máximo e mínimo → **jitter limitado**). Um **fluxo (stream)** é uma comunicação orientada a conexão com transmissão **isócrona**; é unidirecional, com uma origem e um ou mais destinos (*sinks*), sendo *simples* (um fluxo, ex.: áudio) ou *complexo* (vários, ex.: áudio estéreo, áudio+vídeo).

Qualidade de Serviço (QoS)

A QoS de um fluxo é especificada por métricas como: o **bit rate** de transporte, o **atraso de setup** (até poder começar a enviar), o **atraso ponta-a-ponta**, o **round-trip** e a variação desse atraso — o **jitter**. Para cumprir a QoS, a rede oferece *serviços diferenciados* (priorizar pacotes); além disso, usam-se **buffers** no receptor para absorver o jitter:



Buffer para reduzir o jitter: os pacotes saem regulares da origem, chegam irregulares (jitter) e, após um "tempo no buffer", são reproduzidos regularmente — se um pacote (o 8) chega tarde demais, ocorre uma lacuna no playback.

Perda de pacotes: interleaving

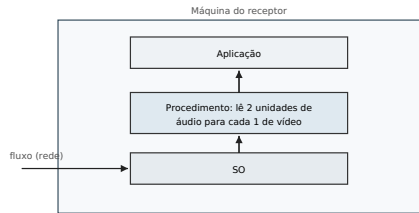
Quando várias amostras vão num mesmo pacote, perder um pacote causa uma **lacuna contígua** de quadros. O **interleaving** intercala as amostras de vários quadros entre os pacotes: assim, a perda de um pacote vira várias **pequenas lacunas espalhadas** (quadros isolados), muito menos perceptíveis:



(a) Sem interleaving, perder o pacote com os quadros 9–12 abre um buraco contíguo. (b) Com interleaving, o mesmo pacote perdido tira quadros isolados (3, 7, 11, 15), espalhando o dano.

Sincronização de fluxos

Num fluxo complexo é preciso **sincronizar** os subfluxos — por exemplo, os dois canais de um som estéreo devem diferir menos de **20–30 μs**. A alternativa prática é **multiplexar** todos os subfluxos num único fluxo e **demultiplexá-los** no destino, tratando a sincronização nos pontos de (de)multiplexação (é o que o MPEG faz):

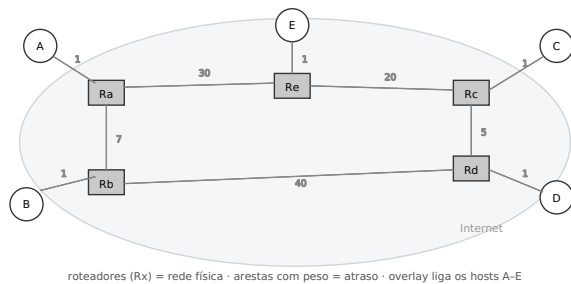


Sincronização por multiplexação: no receptor, um procedimento lê, por exemplo, 2 unidades de áudio para cada unidade de vídeo, mantendo os subfluxos alinhados antes de entregar à aplicação.

6. Multicast

No **multicast em nível de aplicação**, os nós são organizados numa rede *overlay* pela qual os dados são disseminados. A árvore pode ser construída com o **Chord**: o iniciador gera um identificador mid; faz lookup por `succ(mid)`, que se torna a **raiz**; um nó P que queira entrar envia um *join* à raiz, e cada nó Q no caminho que ainda não conhecia a árvore vira **forwarder** (P vira seu filho) e repassa o *join* — até alcançar quem já faz parte da árvore.

Como o multicast roda por cima da rede física, surgem dois **custos** importantes:



Overlay de multicast sobre a rede física (roteadores Rx, arestas com peso = atraso). overlay liga os hosts A-E

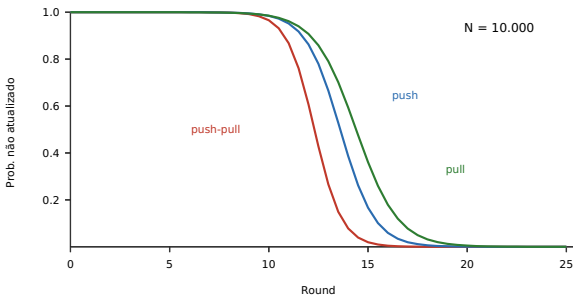
- **Stress no link:** quantas vezes a mesma mensagem de multicast passa pelo *mesmo enlace físico*. No exemplo, uma mensagem de A para D precisa atravessar (Ra, Rb) **duas vezes**.

- **Stretch:** razão entre o atraso pelo caminho do multicast e o atraso pelo caminho da rede. No exemplo, de B para C o caminho do multicast tem tamanho 73, mas existe um de 47 na rede → **stretch = 73/47**.

7. Protocolos epidêmicos (gossip)

Assumindo que não há conflitos *write-write* (uma simplificação forte), uma atualização feita num único servidor é propagada de forma **lazy** (não imediata) aos vizinhos, até que *eventualmente* alcance todas as réplicas. Duas formas de epidemia:

- **Anti-entropy:** cada réplica escolhe periodicamente outra ao acaso e uniformiza estados — por *push* (P só envia a Q), *pull* (P só recebe de Q) ou *push-pull* (trocam entre si). Cada rodada de push-pull dissemina em **$O(\log N)$** rodadas para os N nós.
- **Gossiping:** uma réplica recém-atualizada ("contaminada") repassa a atualização a alguns vizinhos; se contactar alguém que já sabia, ela para de espalhar com probabilidade **$1/k$** .



Desempenho do anti-entropy (N = 10.000): a probabilidade de um nó ainda não ter recebido a atualização despenca por volta das rodadas 12-18. push-pull é o mais rápido; pull tem cauda ligeiramente maior.

Para o gossiping, com muitos servidores a fração s que ainda não recebeu a atualização satisfaz a equação abaixo. A tabela (para $N = 10.000$) mostra que, mesmo aumentando k , sempre restam alguns nós sem update — ou seja, **gossiping sozinho não garante** que todos sejam atualizados:

$$s = e^{-(k+1)(1-s)}$$

k	s (fração sem update)	N·s (nós sem update)
1	0,2032	2032
2	0,0595	595
3	0,0198	198
4	0,0070	70
5	0,0025	25
6	0,0009	9
7	0,00034	3

Remoção de valores

Há um problema intrínseco: não dá para simplesmente apagar um valor e esperar que a remoção se propague — pior, um protocolo epidêmico pode **desfazer** a remoção rapidamente (uma réplica que ainda tem o valor o reintroduz). A solução é registrar a remoção como um tipo especial de atualização, o **atestado de óbito (death certificate)**. Surge então a questão de como remover o próprio atestado (ele não pode ficar para sempre): ou se roda um *algoritmo global* que detecta que todos perceberam a remoção, ou se associa um **tempo de vida máximo** ao atestado. Como a remoção precisa mesmo alcançar todos os servidores, há um problema de escalabilidade: quanto mais servidores, maior o tempo de propagação.

Aplicações

Duas aplicações centrais: **disseminação de dados** (provavelmente a mais importante) e **agregação**. Na agregação, cada nó i mantém uma variável x_i ; quando dois nós fazem gossip, ambos fazem $x_i, x_j \leftarrow (x_i + x_j)/2$. Ao final, todos convergem para a **média** $\bar{x} = (\sum x_i)/N$. Um truque elegante: se um único nó começar com $x = 1$ e todos os demais com 0, a média final será $1/N$ — logo cada nó pode **estimar o número de nós** do sistema calculando $1/x_i$.